Machine Learning Fundamentals

LINEAR REGRESSION

Andy Louwyck
andy.louwyck@vives.be
Academic year 2024 - 2025

hogeschool vives

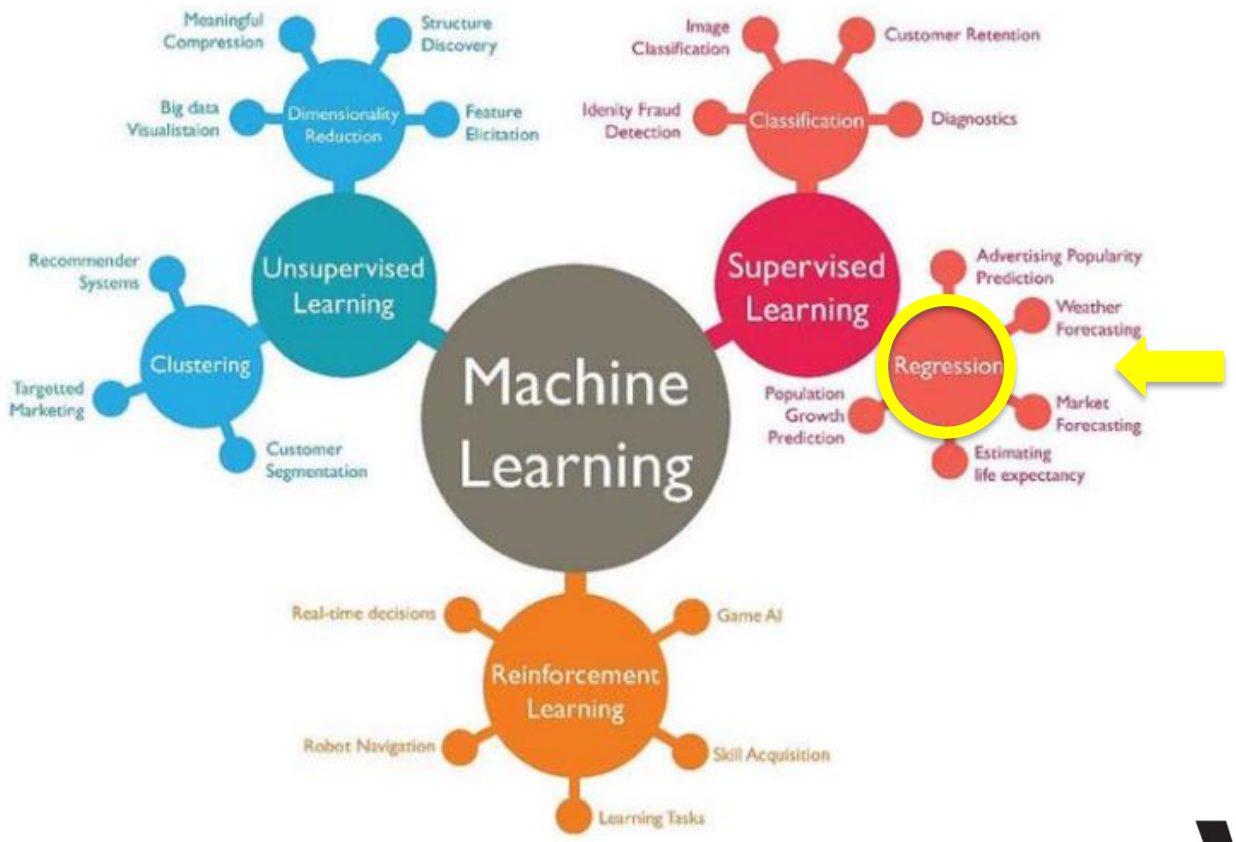
Different types of Machine Learning

What are the three big families?

What's the difference between supervised and unsupervised learning?

What's the difference between classification and regression?

The 3 big families



Category of machine learning. Image by https://www.techleer.com/articles/203-machine-learning-algorithm-backbone-of-emerging-technologies/

hogeschool vives

Data, examples, features, and target

- **Data** = information = table
- **Example** = sample = instance = data point = table row/record
- **Feature** = independent variable = table column/attribute
- **Target** = labels = dependent variable = feature we want to predict

data(set)

	A	B	C	D	E	F
1	id	date	size	typos	recipients	spam
2	0	12/01/2021	2.5	0	1	False
3	1	13/01/2021	1.3	0	2	False
4	2	14/01/2021	12.1	3	15	True
5	3	15/01/2021	7.8	2	19	True
6	4	16/01/2021	4.6	1	5	False
7	5	17/01/2021	9.8	5	1	True
8	6	18/01/2021	11.6	3	63	True

example

feature **target**

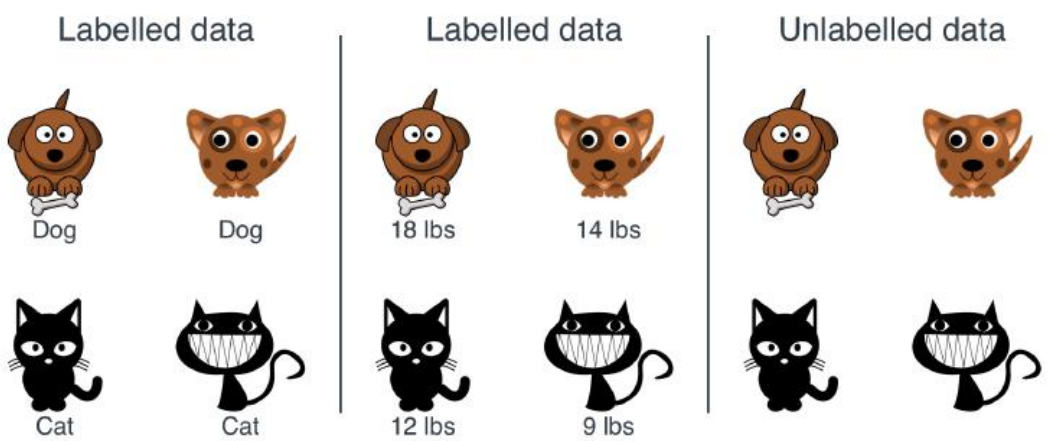
Labeled and unlabeled data

- Labeled data: data with a label

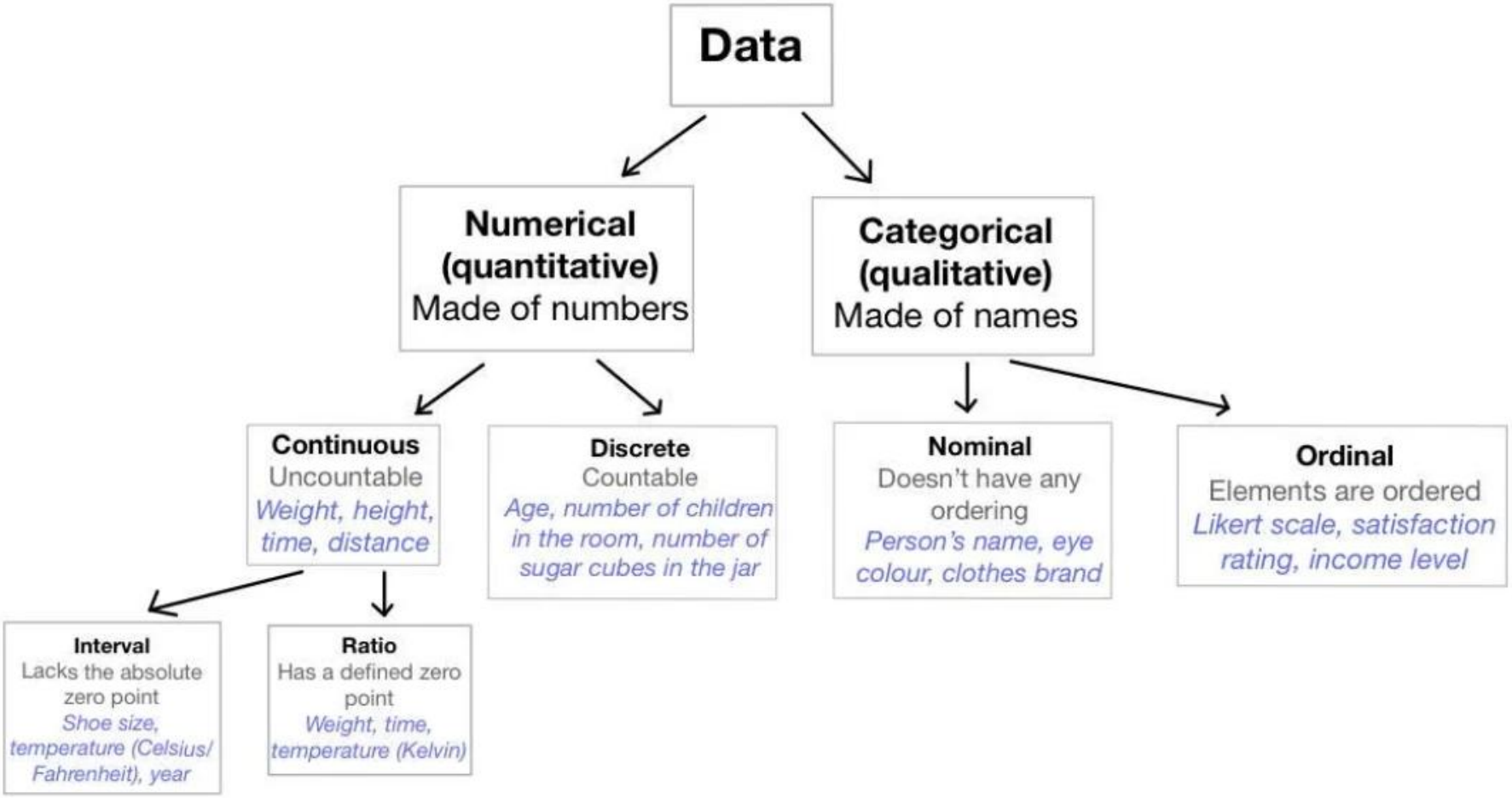
  → **SUPERVISED LEARNING**

- Unlabeled data: data without a label

  → **UNSUPERVISED LEARNING**

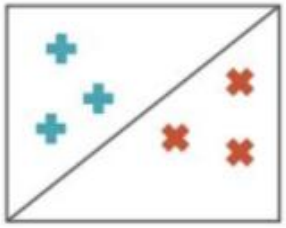


Numerical and categorical data



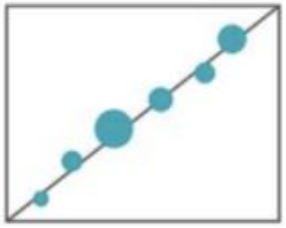
Classification and regression

- **Classification:**
  - Models predicting a **categorical variable**
  - Model output is discrete: a category, a class, a state, …
  - Examples: spam or ham, images with cat or dog, MNIST numbers
- **Regression:**
  - Models predicting a **numerical variable**
  - Model output is (usually) continuous: a real number
  - Examples: predicting house prices, linear regression



CLASSIFICATION

Sorting items into categories



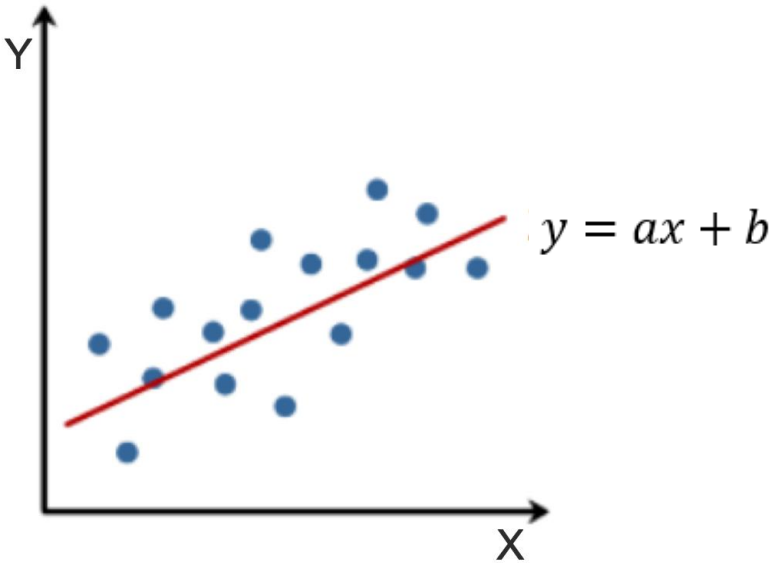
REGRESSION

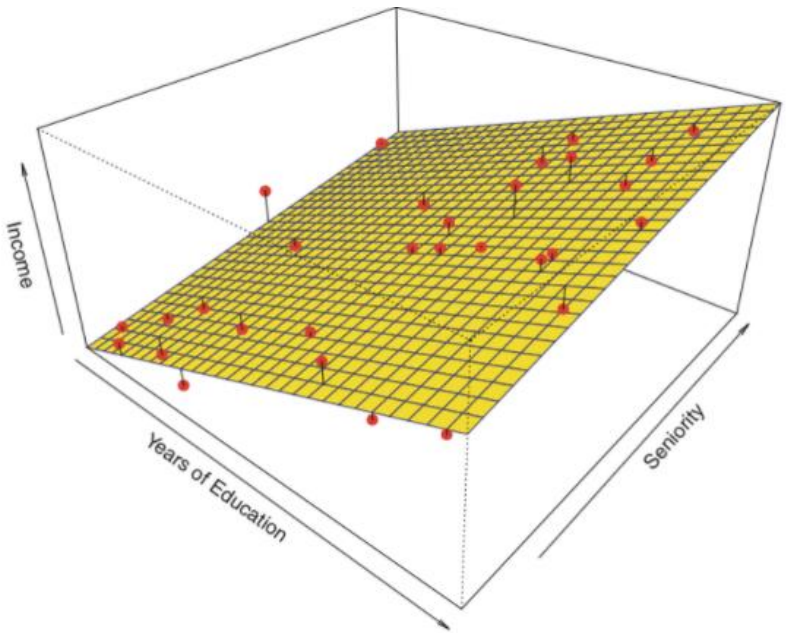
Identifying real values (dollars, weight, etc.)

hogeschool vives

Linear regression

= finding the best-fitting straight line (2D), plane (3D), or hyperplane (ND)





Gradient of a function

What is a gradient?

What is a linear relationship?

What is the use of the gradient?

What is a gradient?

How does one quantity vary as a function of another quantity?

- For example, color gradient:

- For example, speed: distance per unit of time

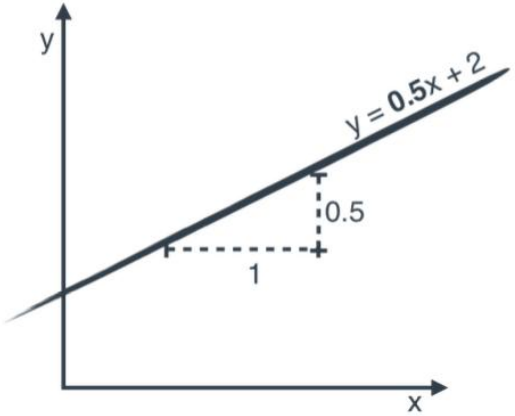
$$speed = \frac{distance_2 - distance_1}{time_2 - time_1}$$

Linear relationship

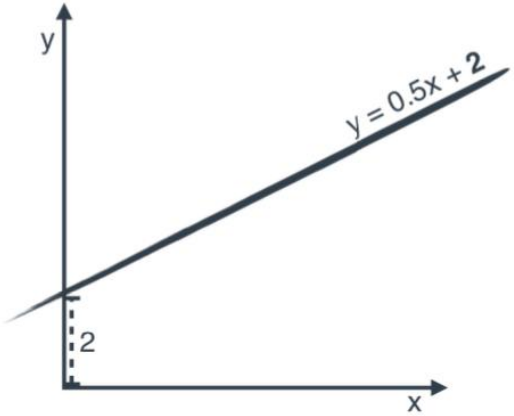
= constant gradient

In 2 dimensions:

- straight line with equation $y = ax + b$
- slope a
- intercept b



Slope = 0.5



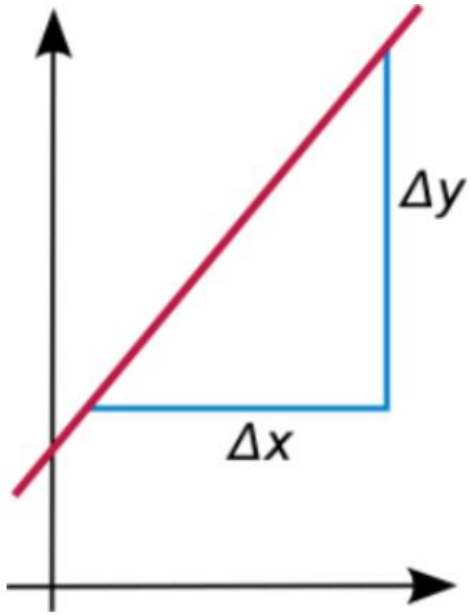
y-intercept = 2

Derivative

In 2 dimensions:

- gradient = derivative
- gradient = slope a

$$y'(x) = \frac{dy}{dx} = a$$



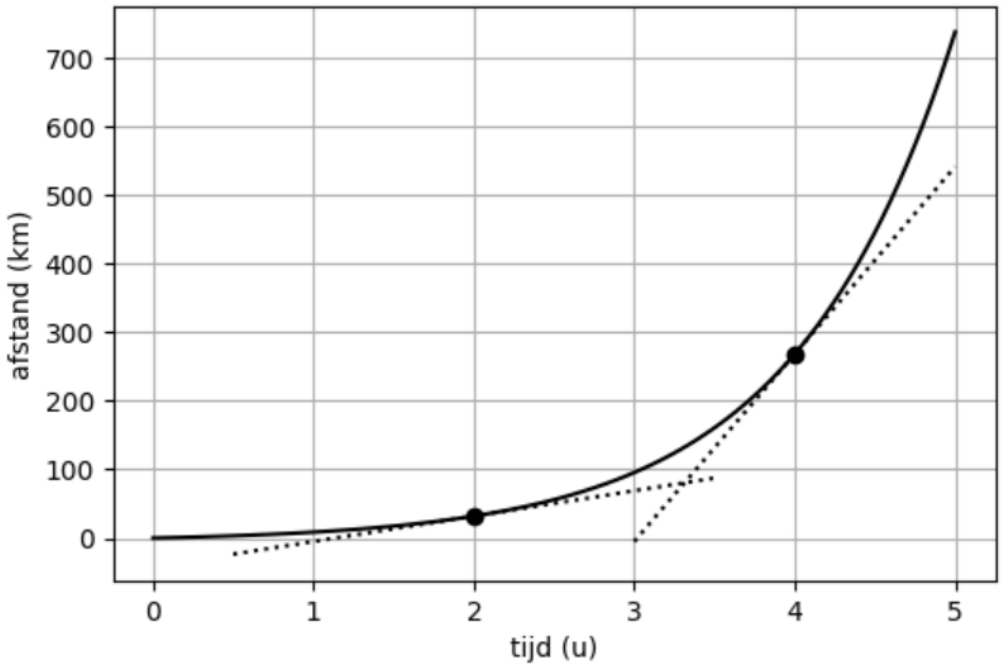
$$\frac{dy}{dx} = \frac{\Delta y}{\Delta x}$$

Non-linear relationship

= gradient is not constant

In 2 dimensions:

- curved instead of straight
- gradient = slope of tangent

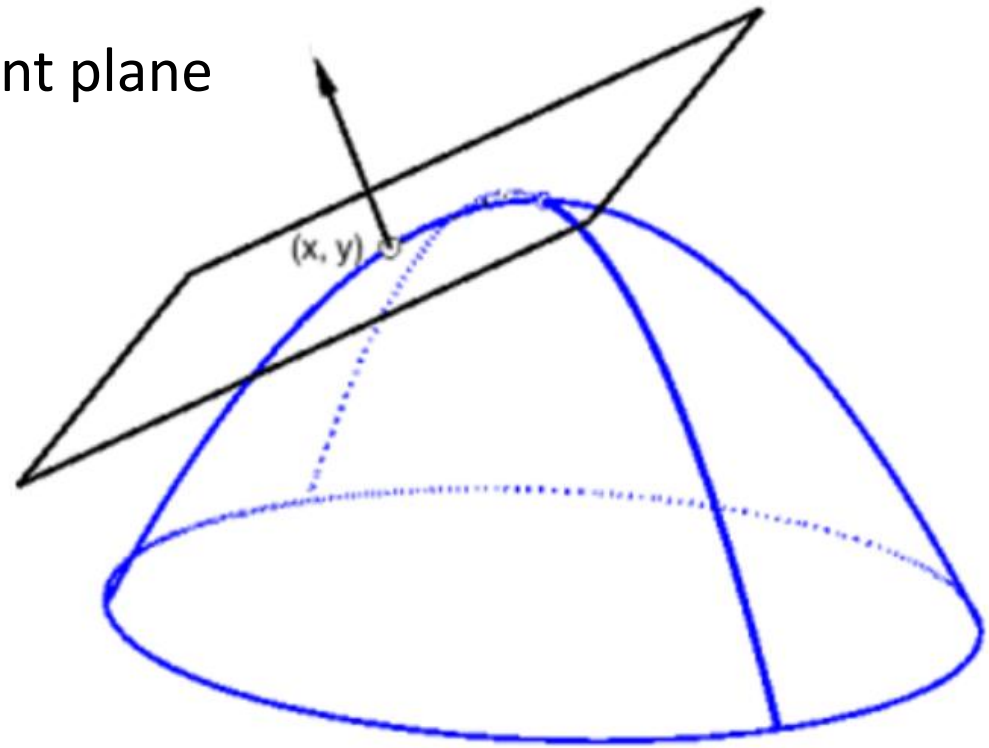


Gradient in multiple dimensions

= generalization of the concept of derivative

In 3 dimensions:

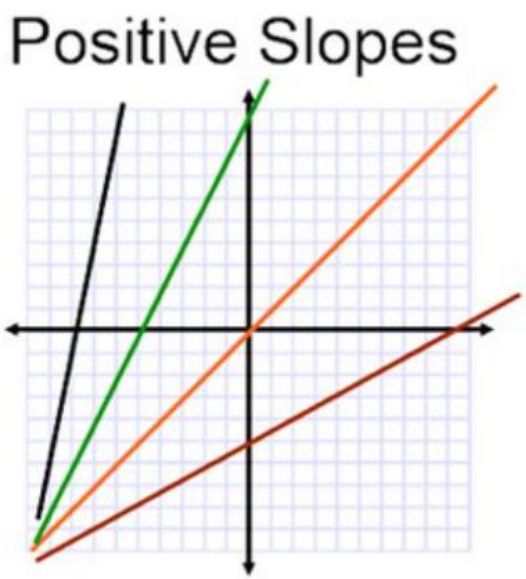
- gradient = slope of tangent plane

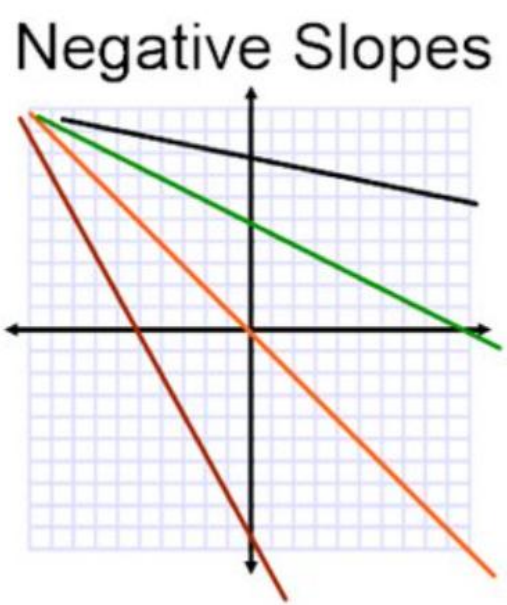


Meaning of the gradient

A gradient indicates:

- whether a curve/surface is rising or falling at a point
- how much a curve/surface rises or falls at that point





Simple linear regression

What is linear regression?

Loss function

Ordinary Least Squares

What is linear regression?

- Predicting target y_i from a linear relationship between features x_i:

$$y_i = a_0 + a_1 x_{i1} + a_2 x_{i2} + \ldots + a_N x_{iN}$$

- For example, in Excel: $a_0 = 3;\ a_1 = 2;\ a_2 = -3;\ a_3 = 0.5$

	x1	x2	x3
i = 1	1	0	10
i = 2	3	2	20
i = 3	-3	5	40
i = 4	4	-4	30

2*x1	-3*x2	0.5*x3
2	0	5
6	-6	10
-6	-15	20
8	12	15

y = 3 + 2*x1 - 3*x2 + 0.5*x3
10
13
2
38

- Machine Learning: find parameters a_k using samples x_i and y_i

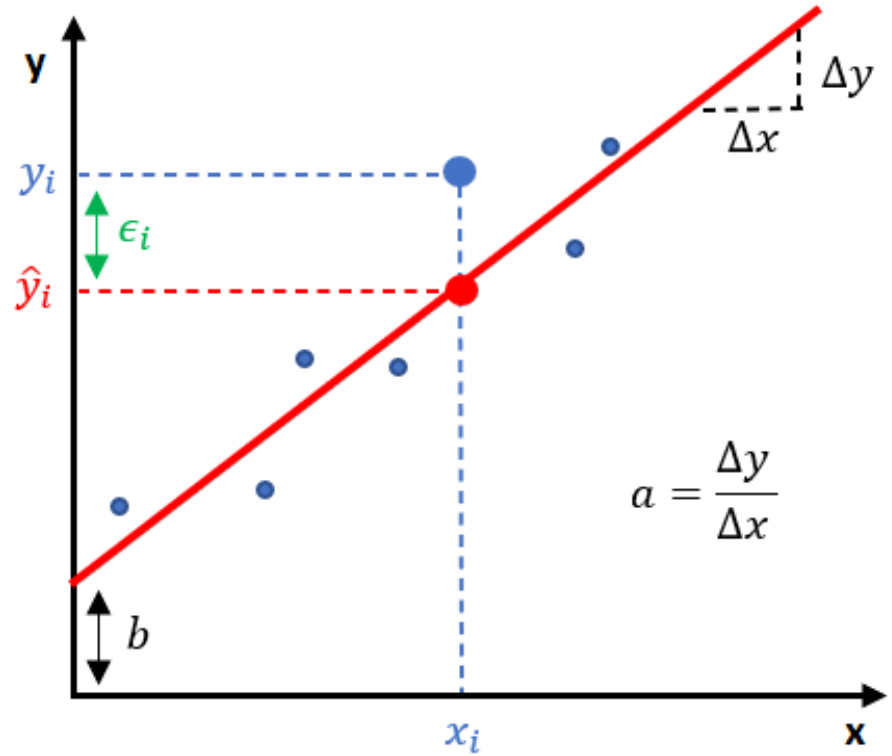
What is simple linear regression?

Linear regression with only 1 feature *x*: $y_i = ax_i + b$

→ Algorithm needed to find *a* and *b*:

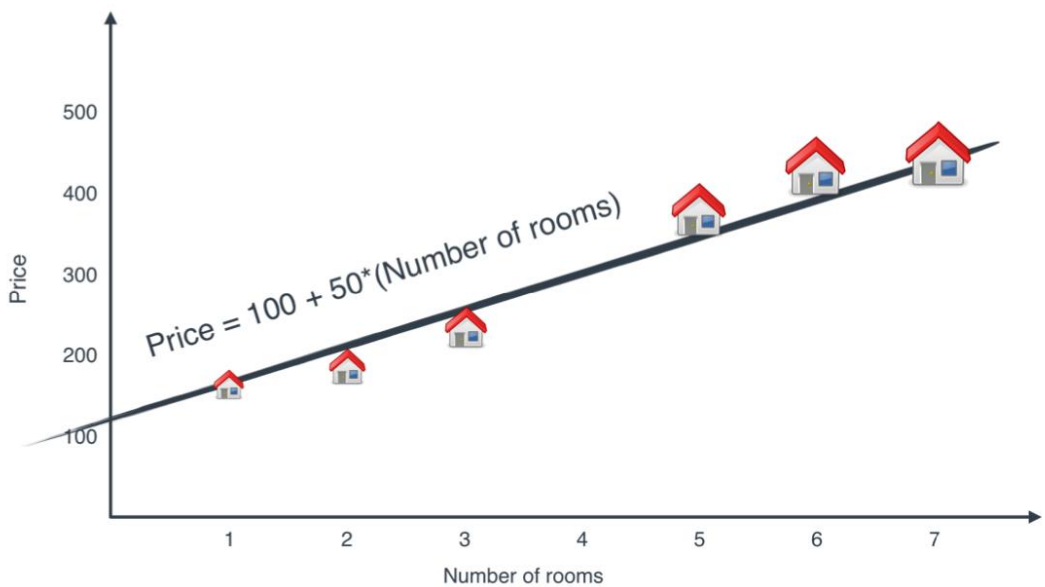
- prediction: $\hat{y}_i = ax_i + b$
- target: $y_i = ax_i + b + \epsilon_i$
- deviation: $\epsilon_i = y_i - \hat{y}_i$

Deviation of all points should be as small as possible!



Example

Equation of straight line: **y = ax + b**
where y is the price, a is the slope
x is the number of rooms, b is the intercept

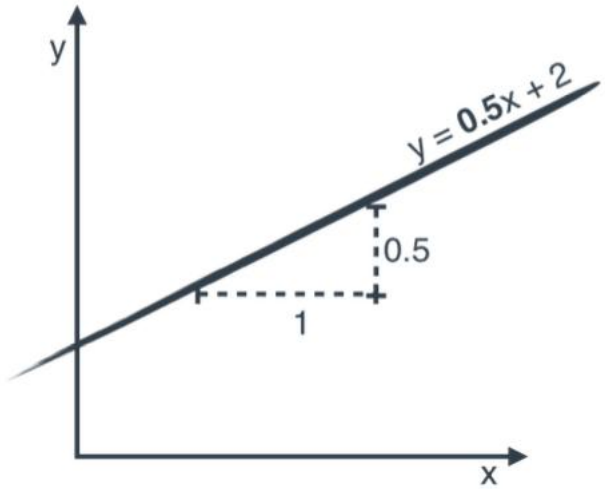


Number of rooms	Price
1	155
2	197
3	244
4	?
5	356
6	407
7	448

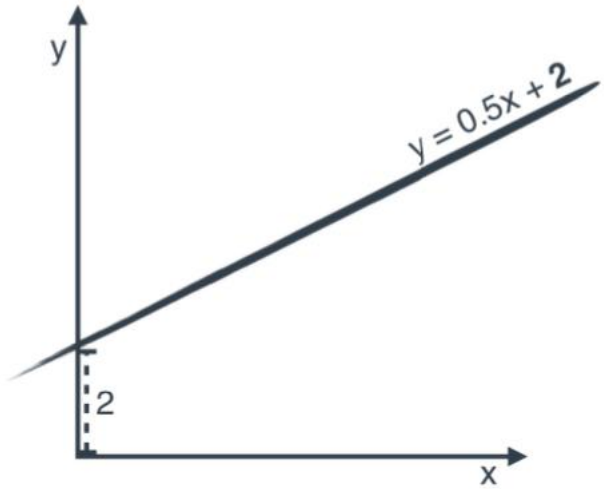
Predicted price for 4 rooms: price = 100 + 50 x 4 = 300

Slope and intercept

Equation of straight line: y = ax + b



Slope = 0.5



y-intercept = 2

We need to find the “slope” and “intercept” that best fits the data

→ a (=slope) and b (=intercept) are the parameters that we optimize

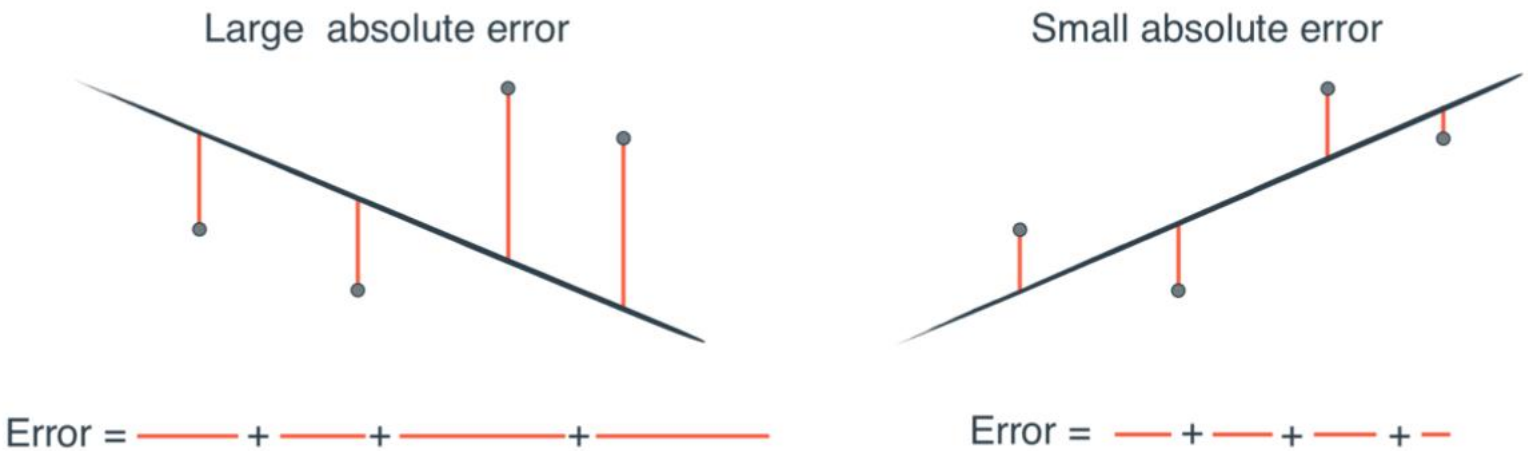
Error or Loss Function (1)

= measure of the deviation or error of all points

Take absolute value because + and – cancel each other out

→ Sum of the absolute value of the deviations

→ Or the average absolute deviation = Mean Absolute Error (MAE)



Error or Loss Function (2)

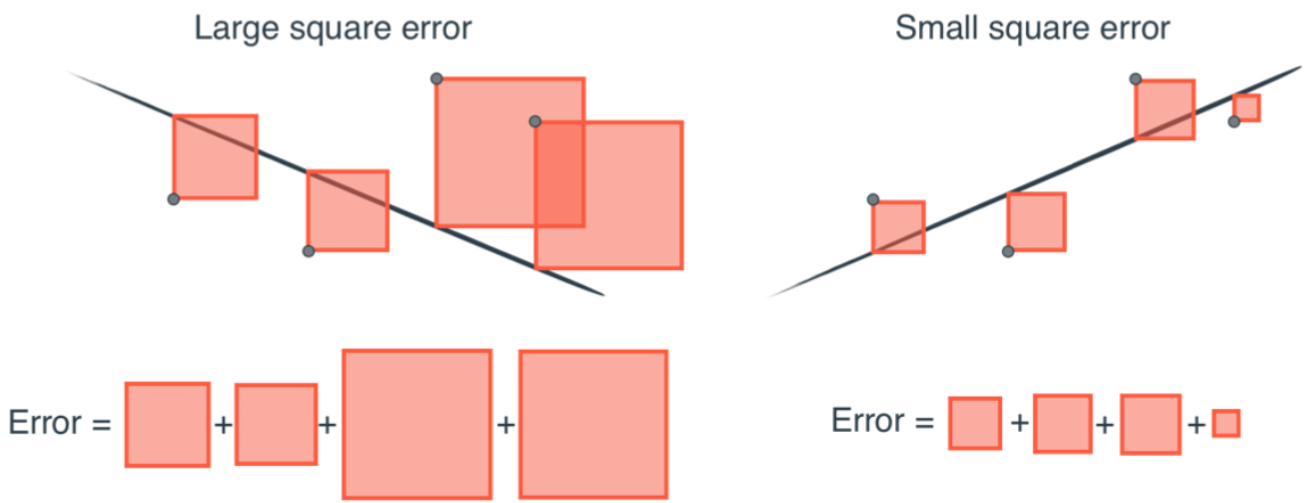
= measure of the deviation or error of all points

Take square because + and – cancel each other out

+ derivative of square is easier

→ Sum of Squared Errors (SSE)

→ Or the mean squared deviation = Mean Squared Error (MSE)



Optimization problem

Derive *a* and *b* by minimizing the loss function *L*

Gradient of *L* equals zero: $\frac{\partial L}{\partial a} = 0$ and $\frac{\partial L}{\partial b} = 0$

- **Exact solution** exists for simple linear regression by applying the ordinary least squares method

$$b = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

$$a = \bar{y} - b\bar{x}$$

- **Iterative Method**: Gradient Descent

Gradient Descent

How to minimize the loss function?

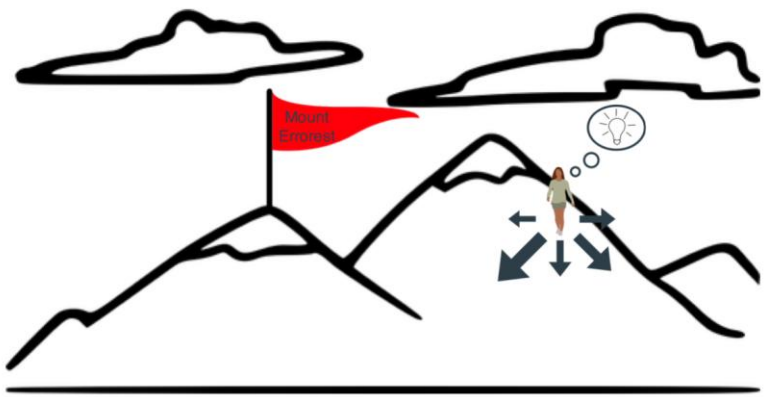
What is the role of the gradient?

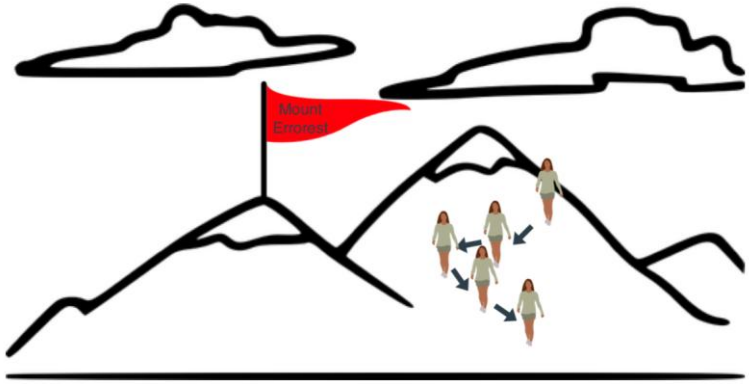
The algorithm

Terminology

Gradient Descent: Analogy

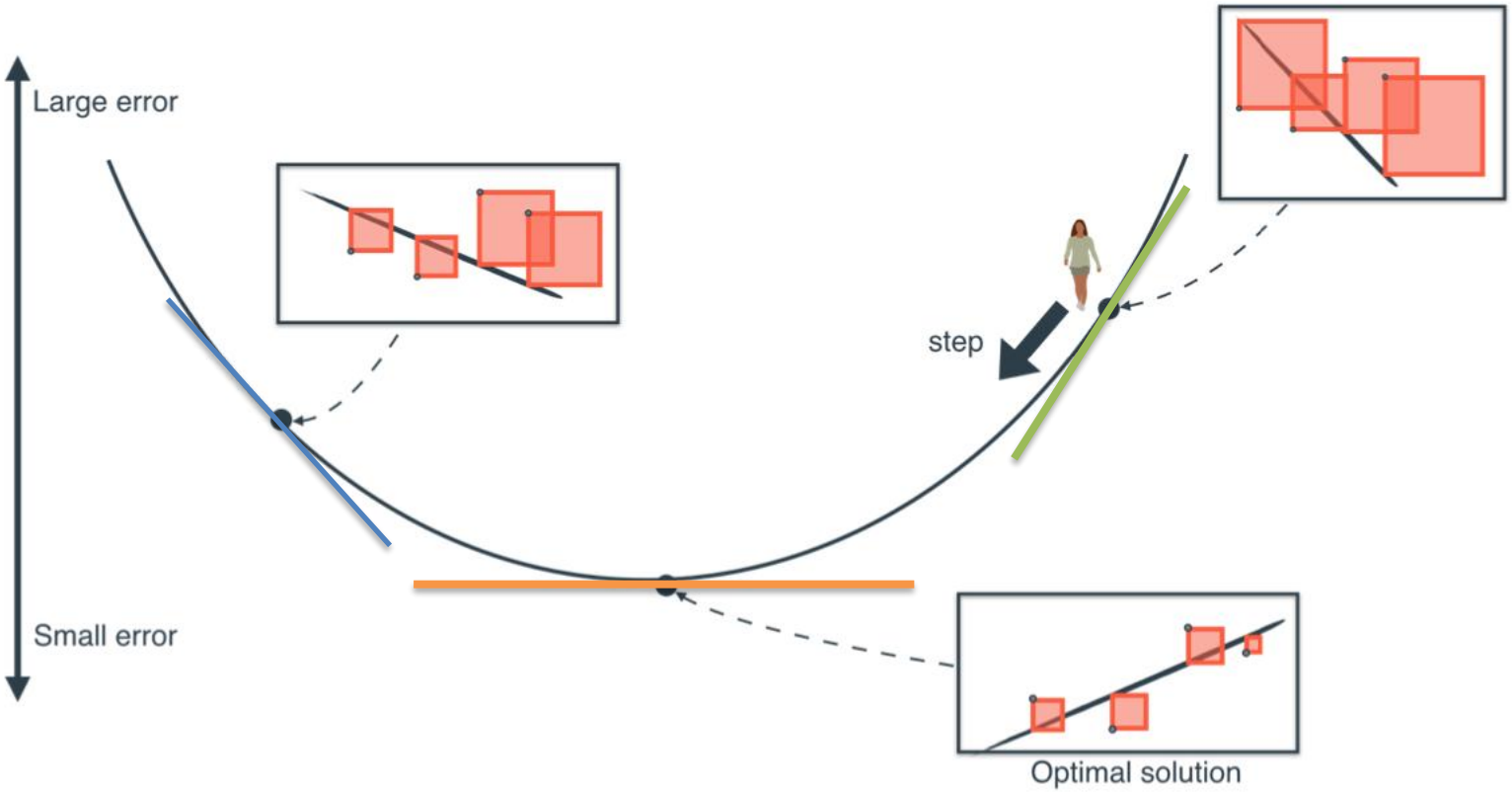
- Loss function can be represented by a mountain landscape = error plane
- How does gradient descent work?
  - You are standing on a mountain somewhere = place with a big total error
  - You want to get to the deepest valley as quickly as possible = place with the smallest possible error
  - You take a step in the direction where the slope is steepest = greatest gradient
  - You keep taking steps (= iterating) until you can't go any further down
  - Hopefully this will help you reach the deepest valley!





Role of the gradient

- Gradient = the derivative of the error function
  = slope of the tangent line
- Minimum reached? Gradient = 0!



Gradient Descent in words

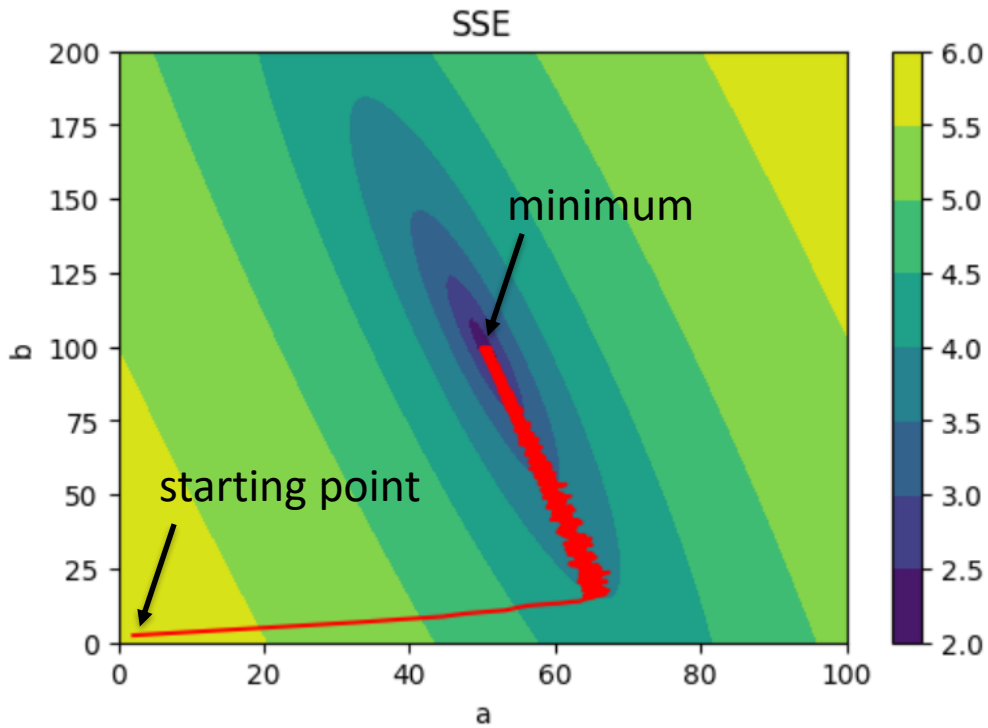
- Loss function can be represented by a mountain landscape = error plane
- How does gradient descent work?
  - You are standing on a mountain somewhere = place with a big total error
  - You want to get to the deepest valley as quickly as possible = place with the smallest possible error
  - You take a step in the direction where the slope is steepest = greatest gradient
  - You keep taking steps (= iterating) until you can't go any further down
  - Hopefully this will help you reach the deepest valley!

- Loss function is e.g. the SSE
  = 2D function of parameters *a* and *b*
- How does gradient descent work?
  - You take random starting values for parameters *a* and *b*
  - You want to find the minimum of the loss function = the smallest possible SSE
  - You calculate the gradient of the loss function at point (*a*, *b*) and adjust *a* and *b* using the gradient so that the SSE becomes smaller
  - You adjust *a* and *b* again and again using the gradient until it becomes (almost) zero
  - Hopefully this will help you find the minimum SSE!

Gradient Descent visualized

2 parameters *a* and *b*:

- error plane is 2D surface (e.g. SSE) → contour plot
- error plane is a function of parameters *a* (x-axis) and *b* (y-axis)
- deepest point is the minimum of the SSE = optimal *a* and *b*



Gradient Descent: The Algorithm

Given:

- a dataset with n data points (x_i, y_i)
- the number of epochs n_{epochs}
- the learning rate α

Steps:

0. Initialize parameters a and b with random values
1. Repeat n_{epochs} times: ← Outer "epoch" loop
    2. Repeat n times: ← Inner "data point" loop
        3. Select a random data point (x_i, y_i) from the dataset (without replacement!)
        4. Compute the prediction $\hat{y}_i = ax_i + b$
        5. Compute the deviation $\epsilon_i = y_i - \hat{y}_i$, and multiply the deviation by the learning rate: $\delta_i = \alpha\epsilon_i$
        6. Adjust parameters a and b on by means of the deviation:
            - $a_{new} = a + \alpha(y_i - \hat{y}_i)x_i = a + \delta_i x_i$
            - $b_{new} = b + \alpha(y_i - \hat{y}_i) = b + \delta_i$

Adjusting *a* and *b* using the gradient

Gradient Descent: Implementation (1)

```python
from random import shuffle

def gradient_descent(x, y, n_epochs, alpha):
    """
    Gradient Descent algorithm for simple linear regression

    Input parameters:
    - x: array with values of the independent variable or the feature (length is n)
    - y: array with values of the dependent variable or the target (length is n)
    - n_epochs: integer that determines the number of epochs
    - alpha: float that determines the learning rate

    Return values:
    - a: array with the slope derived after each iteration (length is n_epochs * n)
    - b: array with the intercept derived after each iteration (length is n_epochs * n)
    - L: array with SSE calculated after each epoch (length is n_epochs)
    The optimal values for slope and intercept are therefore a[-1] and b[-1],
    and the corresponding SSE is L[-1]
    """
```

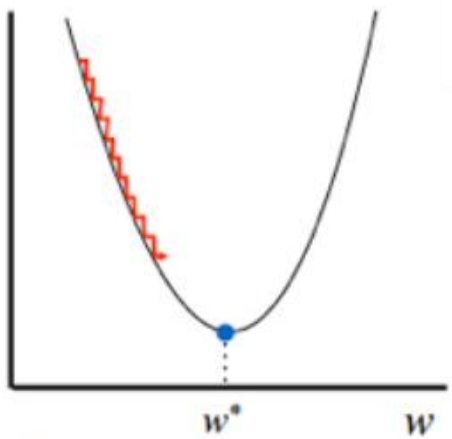
Gradient Descent: implementation (2)

```python
def gradient_descent(x, y, n_epochs, alpha):
    # initialize
    a, b = np.random.rand(), np.random.rand()
    a_path, b_path = [], [] # we are going to keep track of all values of a and b
    L = [] # loss function
    idx = list(range(len(x))) # indices of the data points

    # outer loop: epochs
    for _ in range(n_epochs):

        shuffle(idx) # shuffle indices

        # inner loop: randomize data points
        for i in idx:

            yp = a * x[i] + b  # prediction
            delta = alpha * (y[i] - yp)  # learning rate times deviation

            a += delta * x[i]  # adjust slope
            b += delta  # adjust intercept

            a_path.append(a)
            b_path.append(b)

            # loss function: SSE
            L.append(np.sum((y - (a * x + b)) ** 2))

    # output
    return np.array(a_path), np.array(b_path), np.array(L)
```

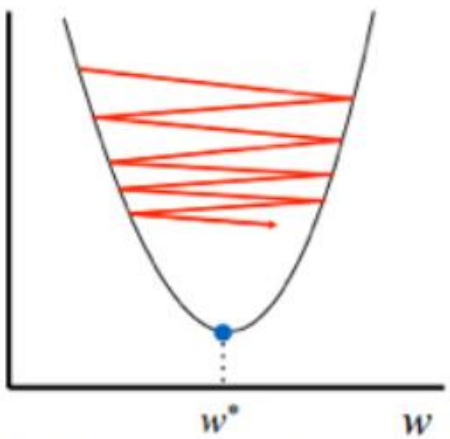
Adjusting a and b using the gradient

Gradient Descent: terminology

- **Parameters** *a* and *b*: derived by the algorithm
- **Hyperparameters**: specified by the user
  - Number of epochs = number of times the dataset is passed through
  - Learning rate α: determines the size of the steps



Too small: converge very slowly



Too big: overshoot and even diverge

Stochastic Gradient Descent

- Gradient of the loss function is approximated each iteration using one data point = **Stochastic Gradient Descent** (SGD)
- Stochastic = algorithm contains 'randomness':
  1. 'random' initial values for parameters *a* and *b*
  2. 'random' traversal of the data points per epoch

- Gradient can also be approximated using multiple data points

  = a **batch**:
    - Batch size = number of points in a batch (= a hyperparameter!)
    - Batches are also randomly composed

  = **Mini-batch Gradient Descent**

Evaluation metrics

MAE and MSE

RMSE

R^2 and R

Evaluation metrics for regression models

= objectively evaluate how well a model predicts

- MAE = mean absolute error
- MSE = mean squared error

- RMSE = root mean square error

  = square root of MSE

  → same unit as the target!

Evaluation metrics for simple linear regression

How strong is the linear relationship?

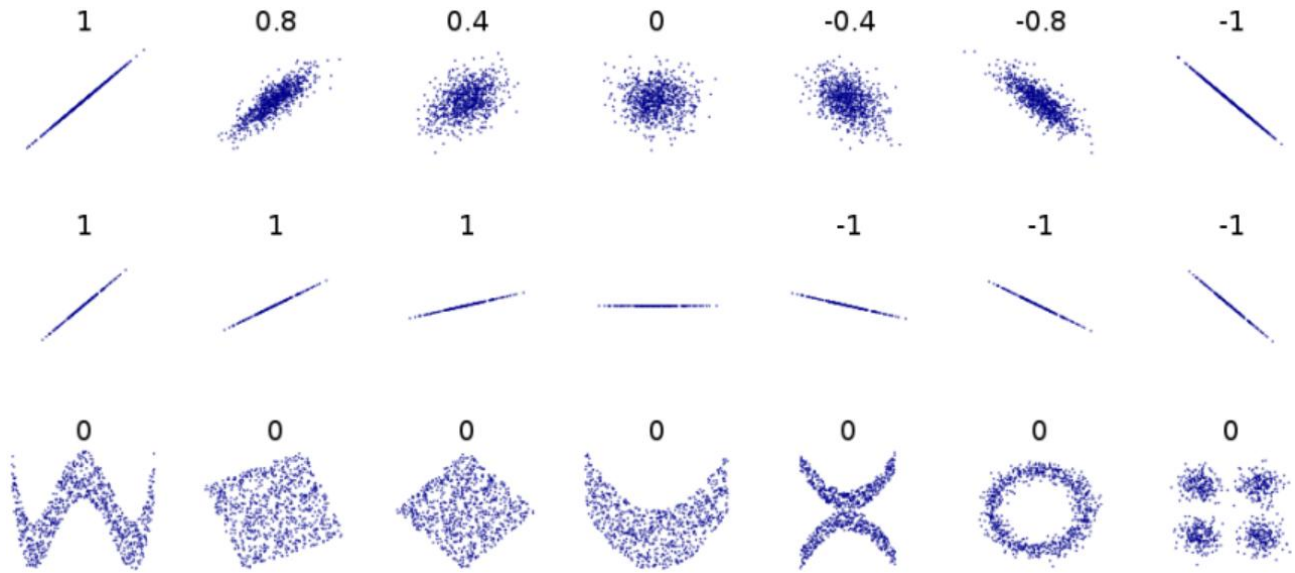
- R^2 = **coefficient of determination**

  = number between 0 and 1: the closer to 1, the better the model

- R = **Pearson's correlation coefficient**

  = square root of R^2, so a number between -1 and 1

CAUTION! Correlation ≠ causation!



No linear relationship
Well other relationship!

Scikit Learn

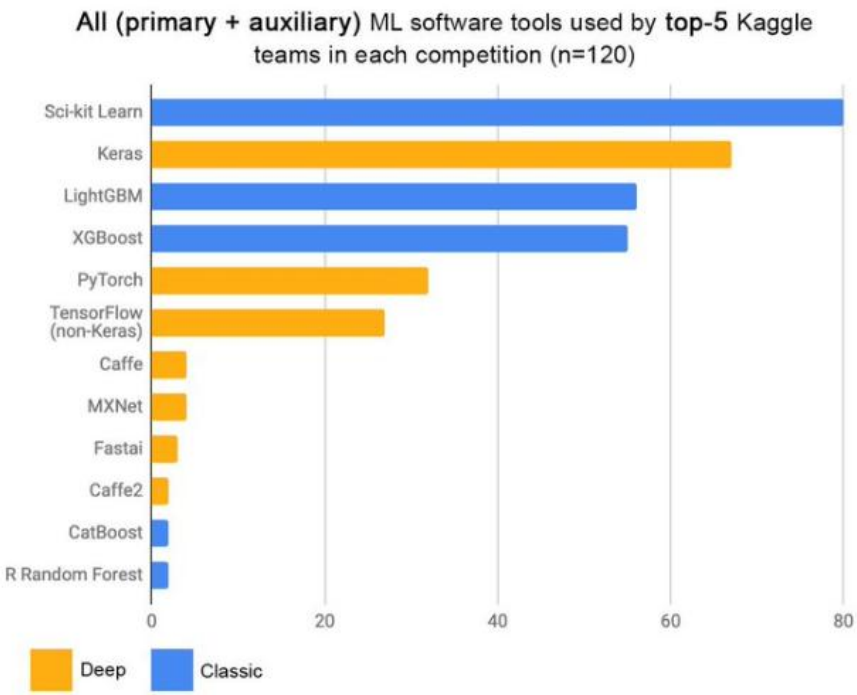
What is Scikit Learn?

How to use for applying linear regression?

What is Scikit Learn?

= free open source Python library for machine learning

= most popular library for “shallow learning”



François Chollet @fchollet · Apr 3, 2019

For the record, we got 59 data points from 2018, 48 from 2017, and 13 from 2016 -- the data is very contemporary.

Linear regression

- Create LinearRegression object and call fit method:

```
from sklearn.linear_model import LinearRegression

x, y = dataset.x.values, dataset.y.values

lr = LinearRegression()
lr.fit(x[:, np.newaxis], y)
```

- Get parameters:

```
print(f"slope = {lr.coef_[0] :.2f}, intercept = {lr.intercept_ :.2f}")
```

```
slope = 50.39, intercept = 99.60
```

- Making predictions:

```
lr.predict([[4]])
```

```
array([301.16666667])
```

Metrics

- R^2

```
lr.score(x[:, np.newaxis], y)
```

```
0.9979437166148433
```

- MAE and MSE

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

MAE = mean_absolute_error(y_true=y, y_pred=yp)
MSE = mean_squared_error(y_true=y, y_pred=yp)
R2 = r2_score(y_true=y, y_pred=yp)

print(f"The MAE is {MAE:.5f}")
print(f"The MSE is {MSE:.5f}")
print(f"The R² score is {R2:.5f}")
```

```
The MAE is 4.83333
The MSE is 24.41865
The R² score is 0.99794
```

Example: LinkedIn dataset (connections vs salary)

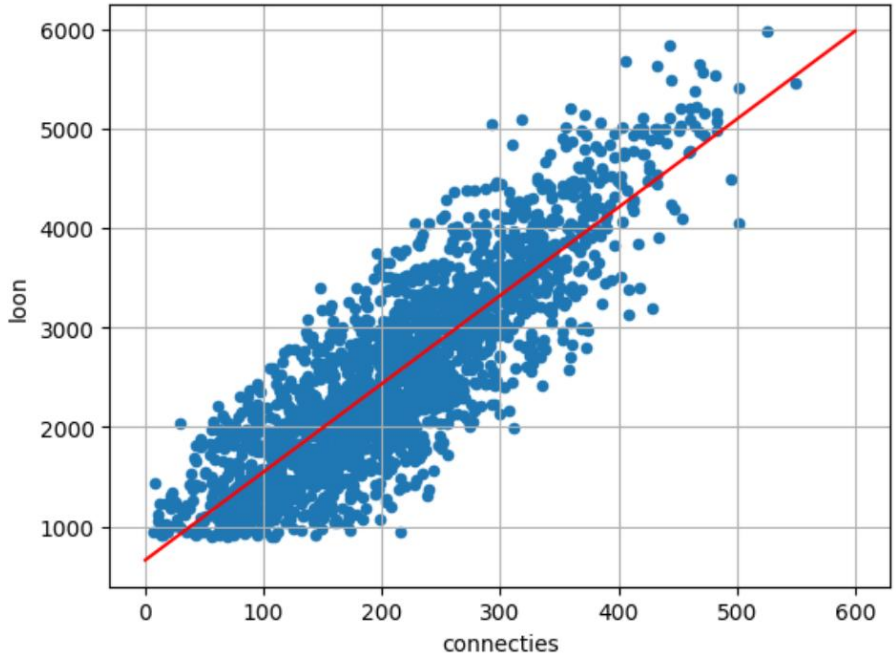
```
from sklearn.linear_model import LinearRegression
LR = LinearRegression()
LR.fit(x.reshape(-1, 1), y)
print(f"slope = {LR.coef_[0]:.5f}\nintercept = {LR.intercept_:.5f}")
yp = LR.predict([[xnew / 600]])[0] * 6000
print(f"The predicted salary for someone with {xnew} connections is {yp:.2f}")
```

```
slope = 0.89131
intercept = 0.10861
The predicted salary for someone with 252 connections is 2897.74
```

Scikit-Learn applies

ordinary least squares (OLS)

= <u>exact</u> solution!



Scaling or normalizing features

- Gradient Descent cannot handle large numbers well
  E.g., LinkedIn dataset: overflow error

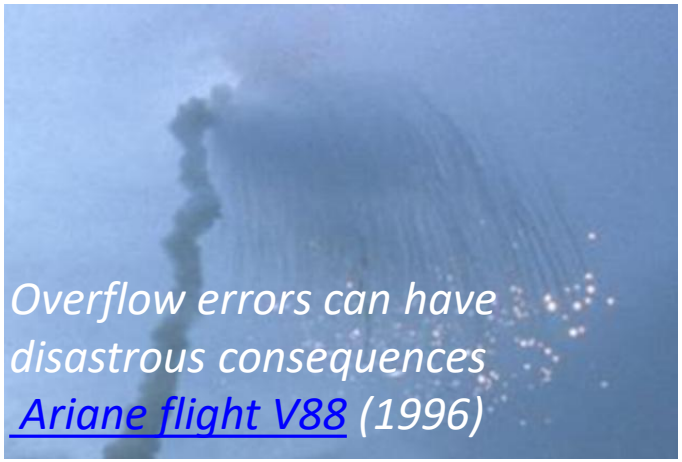
```
# Apply Stochastic Gradient Descent
a, b, L = SGD(x=df.connecties.values, y=df.loon.values,
              n_epochs=10, alpha=0.001, seed=0)
a[-1], b[-1], L[-1]
```

```
<ipython-input-232-e78eb2af9e4c>:39: RuntimeWarning: overflow encountered in scalar multiply
  yp = a * x[i] + b  # prediction
<ipython-input-232-e78eb2af9e4c>:41: RuntimeWarning: invalid value encountered in scalar add
  a += delta * x[i]  # adjust slope
<ipython-input-232-e78eb2af9e4c>:42: RuntimeWarning: invalid value encountered in scalar add
  if fit_intercept: b += delta  # adjust intercept
(nan, nan, nan)
```

- Techniques:

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

  - **Scale**: $\frac{x-\min(x)}{\max(x)-\min(x)}$
  - **Normalize**: $\frac{x-\mathrm{mean}(x)}{\mathrm{std}(x)}$



Keras

What is Keras?

How to train a neural net for linear regression?

What is Keras?

= free open source Python library for deep learning

= high-level API of Google's TensorFlow

From Wikipedia, the free encyclopedia

Keras is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library.

Up until version 2.3, Keras supported multiple backends, including TensorFlow, Microsoft Cognitive Toolkit, Theano, and PlaidML.[1][2][3] As of version 2.4, only TensorFlow is supported. Designed to enable fast experimentation with deep neural networks, it focuses on being user-friendly, modular, and extensible. It was developed as part of the research effort of project ONEIROS (Open-ended Neuro-Electronic Intelligent Robot Operating System),[4] and its primary author and maintainer is François Chollet, a Google engineer. Chollet is also the author of the Xception deep neural network model.[5]

Keras

Original author(s) François Chollet

Linear Regression in Keras
Stochastic Gradient Descent

```
import pandas as pd

linkedin = pd.read_csv("linkedIn.csv")
linkedin.head()
```

	loon	connecties
0	3252	304
1	2968	216
2	2976	159
3	3255	273
4	1953	191

- Rescaling variables

```
x = linkedin.connecties.values / linkedin.connecties.max()
y = linkedin.loon.values / linkedin.loon.max()
```

```
X = x[:, np.newaxis]  # x is a 1D vector -> X is a 2D matrix
Y = y[:, np.newaxis]  # y is a 1D vector -> Y is a 2D matrix
```

- Defining the model

```
from tensorflow import keras
from keras import Sequential, layers

model = Sequential()                # Sequential is a class that we must instantiate
model.add(layers.Dense(units=1))    # using method add() we add a Dense layer with 1 unit
```

Neural net with 1 node only

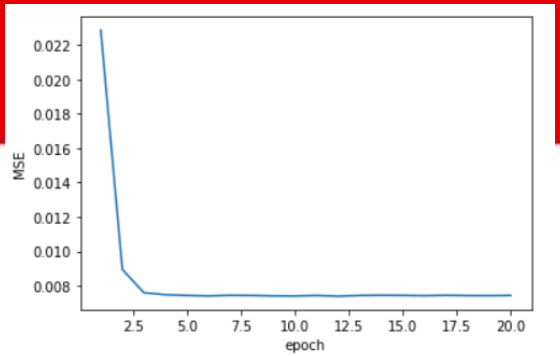
- Compiling the model

Stochastic Gradient Descent

```
sgd = keras.optimizers.SGD(learning_rate = 0.01)
model.compile(optimizer=sgd, loss="MSE")
```

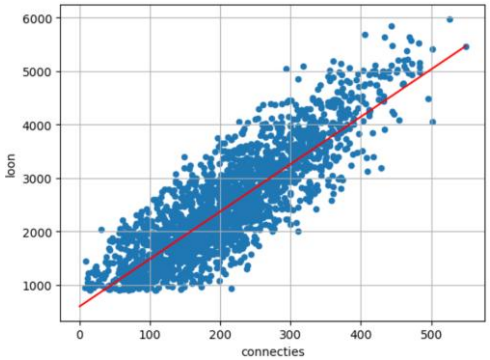
hogeschool vives

Linear Regression in Keras



- Training the model

```
history = model.fit (X, Y, batch_size=1, epochs=20);
```



- Getting the parameters

```
slope = model.weights[0].numpy()[0][0]
intercept = model.weights[1].numpy()[0]
```

- Making predictions

```
x_new = 252 / linkedin.connecties.max()                # don't forget to scale!
y_pred = model.predict(np.array([[x_new]])).item()     # here too we work with 2D arrays
y_pred * linkedin.loon.max()                           # don't forget to scale back!
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
2832.864986240864
```

Sources

- Textbook: Grokking Machine Learning (version 12), Manning Early Access Program, by Luis G. Serrano, 2020.
- Chapter "Correlation and regression", Course Data Science I, Professional Bachelor's Degree in Applied Computer Science, Karel de Grote Hogeschool, compiled by K. Demuynck, 2018.
- https://nl.wikipedia.org/wiki/Gradi%C3%ABnt
- https://nl.wikipedia.org/wiki/Gradi%C3%ABnt_(wiskunde)
- https://en.wikipedia.org/wiki/Least_squares
- http://www.personal.psu.edu/sxj937/Notes/Tangent_Planes_and_Linear_Approximations.pdf
- https://nl.wikipedia.org/wiki/Determinatieco%C3%ABffici%C3%ABnt
- https://nl.wikipedia.org/wiki/Correlationco%C3%ABffici%C3%ABnt
- https://www.knowledgehut.com/blog/data-science/interpret-r-squared-and-goodness-fit-regression-analysis
- https://towardsdatascience.com/batch-mini-batch-stochastic-gradient-descent-7a62ecba642a
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
- https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- "Deep Learning with Python" (2nd Edition) by François Chollet (2021)

hogeschool vives